

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ» (НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет _____

Кафедра _____

Направление подготовки _____

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

(Фамилия, Имя, Отчество автора)

Тема работы _____

«К защите допущена»

Заведующий кафедрой

ученая степень, звание

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

Научный руководитель

ученая степень, звание

должность, место работы

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

Дата защиты: «.....».....20...г.

Новосибирск, 20__

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. Предметная область	6
1.1. Функциональные промежуточные представления	6
1.1.1. Программирование в стиле передачи продолжений .	7
1.2. Императивные промежуточные представления	8
1.2.1. CFG	8
1.2.2. SSA	11
1.2.3. Построение SSA-формы	12
1.2.4. Восстановление SSA-свойства	15
1.2.5. Sea of nodes	17
1.3. Соответствие SSA и функциональным программированием.	18
1.4. Thoring IR	19
2. ПОСТАНОВКА ЗАДАЧИ	21
3. РАЗРАБОТАННЫЙ ПОДХОД	22
4. РЕЗУЛЬТАТЫ	23
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25
ПРИЛОЖЕНИЕ	27
П.1. Первая глава приложения	27

ВВЕДЕНИЕ

Первые компиляторы появились во второй половине двадцатого века и прошли большой путь развития до наших дней. Сегодня компилятор - большая и сложная система, которой приходится мириться с новыми абстракциями языков программирования, появлением управляемых сред исполнения, со сложностью и разнообразием архитектур ЭВМ.

Чтобы такая большая программа как компилятор могла существовать, поддерживаться и развиваться, нужно упростить задачу, которую она решает, например, поделив её на подзадачи. Поэтому процесс компиляции принято делить на фазы, каждая из которых реализуется одним или несколькими программным модулями.

В настоящей работе нас будет интересовать модуль, осуществляющий *машинно-независимые* оптимизации, его мы будем называть *оптимизатором*. Машинно-независимые оптимизации - это трансформации программы, не требующие её отображение в машинный код целевой архитектуры.

Чтобы оптимизатор мог работать с программой ему с необходимостью нужно представить программу в виде некоторой структуры данных. Её мы будем называть *промежуточным представлением*, подчеркивая, что это лишь промежуточная остановка программы в её трансформации от исходника до эффективного машинного кода. Причем наша остановка пусть и удалена от изначальной программы, но все же ближе к ней, чем к языку целевой архитектуры.

Как пользователи оптимизатора, мы хотим, чтобы он работал быстро и порождал эффективные программы. Как разработчики компилятора мы хо-

тим чтобы с ним было удобно работать. Чтобы угодить всем нужно выбрать подходящее промежуточное представление. Как и выбор любой структуры данных, выбор промежуточного представления следует начинать с определения набора операций, выполняемых над этой структурой, и требований к эффективности этих операций. В нашем случае нужно определиться с набором оптимизаций, применимых к программе, относительной частотой этих оптимизаций, необходимых анализов программы. Коль скоро наши оптимизации в меньшей степени зависят от целевого языка, требуемые параметры определяются исходным языком. А языки программирования бывают очень разными.

Языки программирования принято делить по *парадигмам*, то есть по наборам абстракций, который предоставляет язык. Принято выделять *императивные* (к ней мы будем относить и все *объектно-ориентированные* языки) и *функциональные* парадигмы программирования.

Основными абстракциями императивных языков программирования являются *последовательное исполнение выражений*, использование *управляющих конструкций*, наличие *переменных*, разбиение программ на *процедуры* или *функции*. Управляющие конструкции делают поток исполнения программы нетривиальным, чтобы представлять все возможные пути исполнения и анализировать поведения программы вдоль них используется классическое *промежуточное представление* программы в виде *графа потока управления*(англ. *Control Flow Graph, CFG*)[1]. Для упрощения анализа потока данных и некоторых оптимизаций вместе с *CFG* используется *SSA* форма программы[2]. Наследником идей этих представлений является *Море узлов* (англ. *Sea of Nodes, SOF*), которое используется в современных

статических и ЛТ-компиляторах для объектно-ориентированных языков программирования[3,4].

Основными абстракциями функциональных языков программирования являются *функциональные замыкания, функции высших порядков, полиморфные функции, алгебраические типы данных, классы типов*. Языки этой парадигмы, такие как Haskell, SML, Scheme, выбирают в качестве промежуточного представления различные модификации лямбда-исчисления: Административную нормальную форму[5], Continuation Passing Style[6,7]. Так как с помощью хорошо изученных трансформаций термов лямбда-исчисления можно эффективно описывать трансформации и оптимизации функциональных языков.

Современные языки программирования, однако, все более тяготеют к смешению парадигм, абстракции изначально появившиеся в функциональных языках перетекают и активно используются в современных объектно-ориентированных языках. Самым ярким примером такого заимствования являются *функциональные замыкания*, которые можно найти в любом современном императивном языке. Но *промежуточные представления* после подобных заимствований, часто остаются неизменными, и заимствованные концепции выражаются в терминах исходного языка[8–10], что приводит к ухудшению результатов оптимизаций.

Отсюда возникает идея реализации *мультипарадигменного промежуточного представления*. В этой работе содержатся результаты разработки *функционального расширения императивного* промежуточного представления и анализ его эффективности в рамках экспериментального компилятора *Huawei*.

1. Предметная область

1.1. Функциональные промежуточные представления

В качестве *промежуточного представлений* оптимизаторов функциональных языков программирования используются различные модификации *лямбда-исчисления*(1.1).

$$term ::= var \mid (term \ term) \mid (\lambda var. term) \quad (1.1)$$

Классические преобразования термов лямбда исчисления, такие как *β -редукция*, *η -конверсия* подходят для описания большинства оптимизаций функциональных языков. Посмотрим на терм $(\lambda x. 0) (f y)$, применение к нему шага *β -редукции* соответствует удалению избыточных вычислений¹.

Набор подобных преобразований термов в оптимизаторе называется *правилами переписывания* термов. Например правило для удаления лишних вычислений можно записать так

$$(\lambda x. M)N \rightarrow M \text{ } x \notin FV(M)$$

На практике в оптимизаторах часто используется *типизированное лямбда исчисление*, с добавленным связыванием переменных (англ. let-bindings) и сопоставлением с образцом (англ. pattern-matching)[5].

Для знакомства с функциональными *промежуточными представлениями* нам будет достаточно классического *лямбда исчисления* со связыванием

¹ Конечно, такая бета-редукция применима только к ленивым языкам, т.к. вычисление терма $(f y)$ может привести к исключению или к незавершающемуся исполнению.

переменных.

$$(\lambda x. M)N \equiv \text{let } x = N \text{ in } M$$

1.1.1. Программирование в стиле передачи продолжений

Программирование в стиле передачи продолжений (англ. continuation passing style, CPS), это шаблон программирования, позволяющий сделать поток управления программы явным, используя для этого функции высшего порядка, которые мы будем называть *продолжениями* (англ. *continuation*). Будем говорить, что функция f находится в *cps*, если:

1. Вместо того чтобы вернуть значение напрямую принимает дополнительным аргументом продолжение, функцию которая обозначает собой *что нужно сделать* после завершения вычисления. В типизированных языках программирования это требование может быть явно отражено в типе функции: $(t \rightarrow 'a) \rightarrow 'a$, где t - некоторый известный тип, $'a$ - типовая переменная. Получаемая функция полиморфна, ведь возвращаемое значение зависит от того какое продолжение будет передано в функцию.
2. Функция завершается хвостовым вызовом.

Пример использования CPS можно увидеть на листинге 1.1.1. Функции, написанные в стиле передачи продолжений выглядят сложнее и читаются хуже своих классических реализаций. Поэтому использовать их стоит там, где явное выражение потока управления необходимо, например в некоторых фреймворках для асинхронного программирования или в качестве

промежуточных представлений компиляторов.

```
let rec factorial (n: int) (cont : (int -> 'a)) : 'a =  
  if n = 0 then cont(1)  
  else factorial (n - 1) (fun res -> cont(n * res))
```

В том или ином виде CPS используется в большинстве компиляторов популярных функциональных языков программирования. Оптимизаторы `ocamlc`, `orbit`, `rabbit` используют cps в качестве основного внутреннего представления. Элементы cps в виде *join point* используются в `ghc`.

1.2. Императивные промежуточные представления

1.2.1. CFG

Граф потока управления (англ. *Control Flow Graph*) - естественное и простое внутреннее представление императивных программ. Оно хранит не только семантическую информацию о программе, но и явно содержит в себе информацию о всех возможных путях исполнения исходной программы.

В вершинах графа потока управления, называемых *базовыми блоками*, хранится последовательность инструкций, которые выполняются последовательно, без передачи потока управления. Ребра между базовыми блоками означают передачу потока управления, с их помощью можно выразить управляющие конструкции исходного языка, такие как *if*, *for*, *while*, *goto*.

Наиболее простым оптимизациям программы нужна информация лишь о фрагментах кода, например об инструкциях внутри одного базового блока. Если такая оптимизация заметит, что переменная в начале доступного ей набора инструкций является нулем и не изменяется внутри этого набора, то она сможет использовать этот факт и провести ряд трансформаций кода, на-

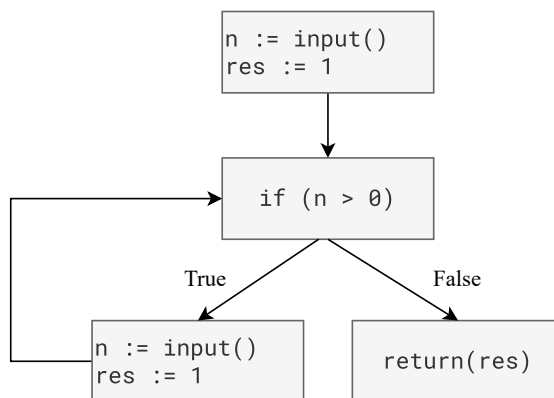


Рис. 1.1: Пример CFG для вычисления факториала

пример заменить инструкции сложения с этой переменной на присваивание. В англоязычной литературе такой класс оптимизаций называются *peephole* оптимизациями, потому что им доступна лишь часть картины мира и это существенно ограничивает их эффективность.

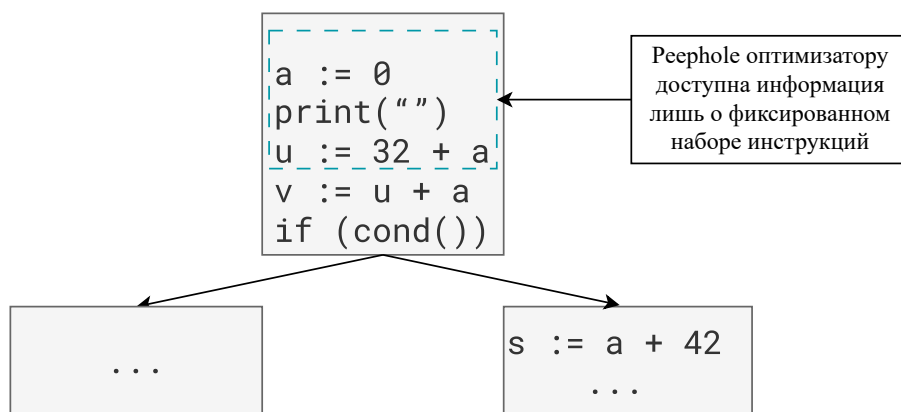


Рис. 1.2: Недостатки реерhole оптимизатора

Явное отображение потока управления в промежуточном представлении открывает возможности для глобального анализа программы, которому доступна информация о данных программы вдоль различных путей исполнения. Такой анализ пересылок данных вдоль дуг потока управления называется *потокowym анализом* и он хорошо изучен. Чтобы сформулиро-

вать задачу потокового анализа нужно записать анализ программы в виде специальных *потоковых уравнений*. Для определенного класса потоковых уравнений, которого хватает для описания множества полезных анализов, существует итеративный алгоритм их решения.

Оптимизации, использующие результаты глобального анализа, превосходят в качестве свои реерhole версии. Например, потоковый анализ может вычислить все участки CFG где переменная тождественна равна нулю, тогда уже знакомая нам оптимизация “сложения с нулем” может свернуть в присваивания все сложения 1.3

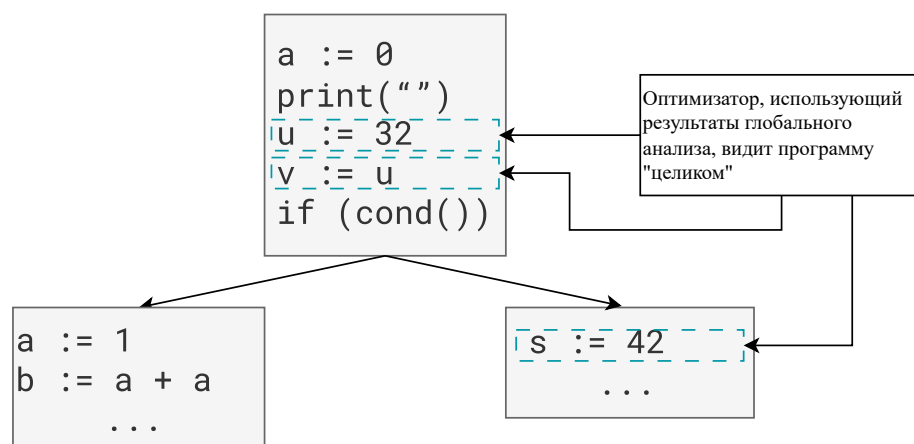


Рис. 1.3: Удаление сложения с нулем, пользуясь результатами потокового анализа

Потоковый анализ используют большинство оптимизаций, работающих с данными внутри программы. Итеративный алгоритм решения потоковых уравнений не всегда сходится быстро, а результаты анализов плохо комбинируются друг с другом.

Одна из наиболее частых задач, встречающихся во время оптимизации - задача нахождения *def-use цепей* переменных, то есть нахождения для каждой точки CFG множества всех использований переменных объявленных в

этой точке и множество всех объявлений переменных, использованных в точке. Такие задачи могут формулироваться как задачи потокового анализа, но при разных условиях будут получаться разные анализы

Проблему нахождения def-use цепи переменной могло бы решить явное присутствие этой информации в IR компилятора, но это затратно как по времени, так и по памяти. Если переменная имеет N объявлений и M использований, то память требуемая для хранения цепи будет пропорциональна $N \cdot M$.

Из-за этих недостатков CFG в своем “чистом” виде редко используется на практике, однако он является основой для более эффективных внутренних представлений.

1.2.2. SSA

Single Static Assignment форма - это промежуточное представление в котором на каждую переменную приходится ровно одна инструкция присваивания. Исследуя выразительные возможности такого представления можно заметить, что написать цикл с индуктивной переменной используя это правило - довольно сложно. Ведь счетчик цикла нужно инициализировать стартовым значением и инкрементировать каждую итерацию. Изобразив граф потока управления для этого случая мы увидим, что проблема происходит в точке схождения потока управления, с одной дуги которого переносится значение стартового значения переменной, с другой обновленное.

Для таких случаев в SSA есть специальные инструкции присваивания, называемые *фи-функциями* или *фи-узлами*. Эта инструкция принимает на

вход несколько значений переменных, и на выходе выдает одно из этих значений, в зависимости от того откуда пришло управление. Фи-узлы всегда располагаются в начале базовых блоков, ведь значение фи-функции должно быть доступно пользователям внутри блока как только управление было передано в блок.

SSA форма упрощает и делает эффективнее множество оптимизаций и анализов, в частности потому что она отлично отображает def-use связи и делает это эффективно.

Листинг 1.1: Простая программа на C в SSA форме.

```
void simple_ssa() {  
    int a_1 = 0;  
    int a_2 = a_1 + 42;  
    fprintf("There a %i\n", &a_2);  
    return a_2;  
}
```

1.2.3. Построение SSA-формы

Построение SSA - это нетривиальная задача. Существует несколько подходов к её решению, здесь мы обсудим лишь два из них.

Классический подход активно использует отношение доминирования на графе: говорят, что вершина CFG v доминирует вершину u (обозначение $v \text{ dom } u$), если любой путь из стартовой вершины в u содержит v . Вершина v строго доминирует u , если $v \text{ dom } u \wedge v \neq u$.

Доминаторы полезны в компиляторах сами по себе, для выражения и проверки некоторых свойств программ, к примеру известное правило, что переменная должна быть объявлена перед использованием в терминах доминаторов формулируется как “объявление переменной должно домини-

ровать все её использования”.

Эффективный алгоритм построения SSA, предложенный исследователями из IBM в 1991 году[11], использует понятие *фронтiera доминирования* для оптимального размещения фи-функций. Для узла v фронтиром доминирования является такое множество вершин u , что v доминирует предка u , но не доминирует u строго.

На рисунке 1.4 синим цветом отмечены узлы, которые доминируются вершиной 4. О фронтире доминирования можно думать как о границе, отделяющей выделенный подграф доминируемых вершин от остального графа. На рисунке фронтиром доминирования вершины 5 будут вершины 3, 9 и exit. Теперь представим, что в 5 есть объявление переменной x , тогда в любой вершине из фронтира достижимы два определения переменной, одно из 5, другое из стартовой вершины (предполагается, что в стартовой вершине все переменные инициализированы начальным значением).

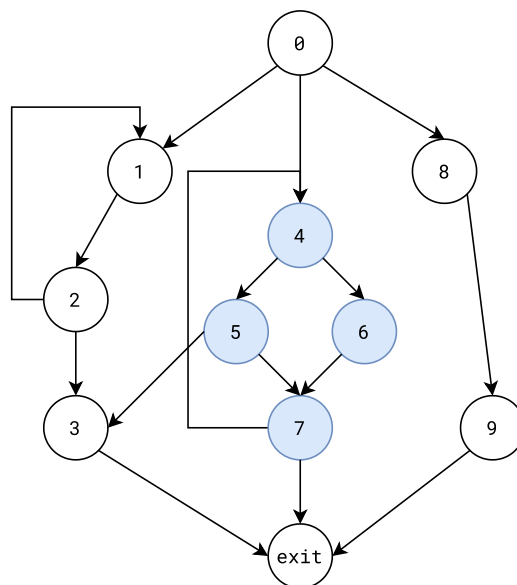


Рис. 1.4: Фронтир доминации

Рассматриваемый алгоритм использует эту интуицию относительно фронтиров для построения *минимальной* SSA-формы. Формальное правило звучит так: если вершина v , содержит определение некоторой переменной x , то для любой вершины из фронта доминирования v требуется фи-функция для x .

Со временем SSA-форма стала основным промежуточным представлением оптимизаторов императивных языков, однако рассмотренный алгоритм её построения работает на уже построенном CFG и требует вычисления информации о доминаторах.

Следующий алгоритм позволяет строить SSA-форму напрямую из AST или байткода. Этот алгоритм был предложен Клифом Кликом в его диссертации [12], однако он порождал не минимальную SSA-форму и мог вставлять фи-функции, значения которых позже в программе не используются. В 2013 году алгоритм был усовершенствован [13], версия из статьи Мэттиаса Брауна строит SSA в котором значение каждой фи-функции используется, а самих фи-функций минимальное количество.

Обсудим идею этого алгоритма. Во время построения IR алгоритм осуществляет локальную нумерацию переменных внутри линейных участков кода. Если же во время этого процесса мы встретим использование переменной, которая не была объявлена внутри линейного участка, то это значит, что объявление содержится в предшественниках текущего участка. Если предшественник всего один, то алгоритм осуществляет рекурсивный поиск объявления в предшественнике. Если же предшественников больше одного, то сначала вставляется новое определение переменной - фи-функция, а затем выполняется рекурсивный поиск определения по

всем предкам участка. Вставка фи-функции перед рекурсией необходима для предотвращения бесконечной рекурсии, ведь поиск по предкам может вновь привести к текущему участку (например при трансляции циклов) и в этом случае мы найдем определение переменной - саму фи-функцию.

Такой алгоритм может вставлять в IR *тривиальные* фи-узлы, то есть такие узлы аргументами которых являются они сами и одно другое значение. Чтобы избежать этого непосредственно перед вставкой фи-узла алгоритм проверяет является ли он тривиальным. Вместе с удалением тривиальных фи-функций предложенный метод также выполняет простейшие data-flow оптимизации такие как упрощение арифметических выражений, удаление общих подвыражений, свертку констант и протяжку копий.

В текущем состоянии алгоритм может конструировать не минимальную SSA-форму для некоторых потоков управления. Но статья описывает дополнительный проход алгоритма, способный эффективно определять набор лишних фи-функций.

Для нас же самое важное в разговоре об алгоритмах построения SSA-формы - это то, что на основании этих алгоритмов можно построить алгоритмы восстановления SSA-свойства программы.

1.2.4. Восстановление SSA-свойства

Оптимизации, изменяющие граф потока управления, часто приводят к инвалидации SSA-свойства IR. Наиболее естественным примером таких оптимизаций являются цикловые оптимизации.

Рассмотрим пример *loop-peeling*'а, оптимизации выносящей из цикла несколько его итераций. Первое что приходит в голову при реализации

этой оптимизации - скопировать тело цикла и вставить его в CFG перед самым циклом.

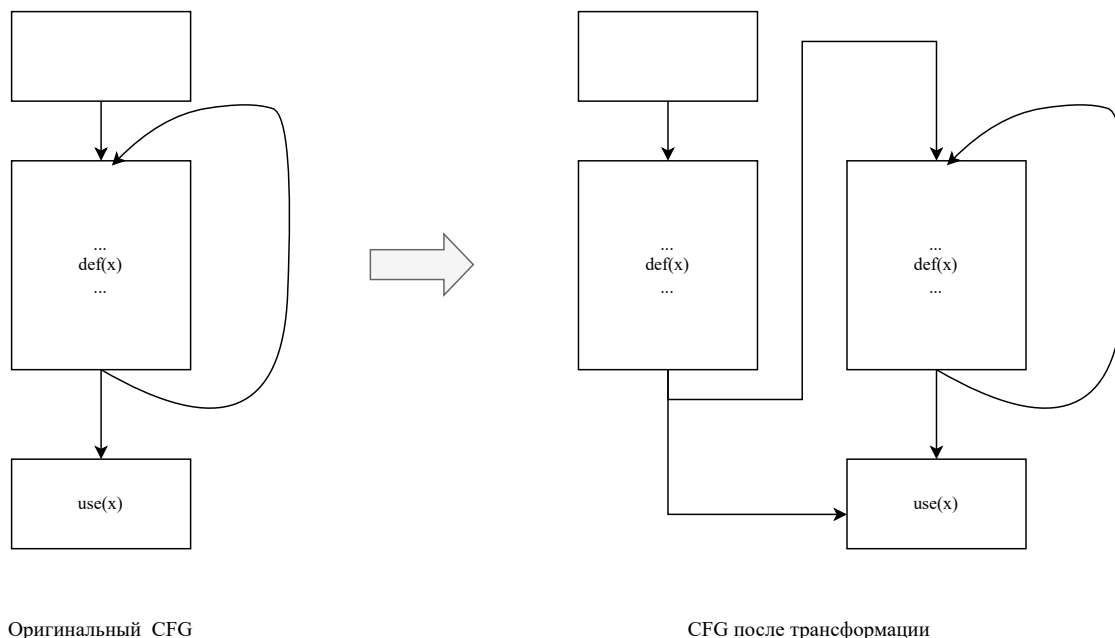


Рис. 1.5: Пример нарушения SSA-свойства копирующего преобразования CFG

Проблема такого решения в том что если тело цикла содержало объявление переменной, которая использовалась вне тела, то наивное копирование добавит второе определение этой же переменной, что нарушит SSA-свойство промежуточного представления.

Для цикловых оптимизаций есть решение этой проблемы - закрытая цикловая SSA-форма. Будем говорить, что цикл находится в закрытой SSA-форме, если переменные объявленные в нем используются только внутри цикла или может быть в базовом блоке, который пост-доминирует тело цикла (выход цикла). В LLVM существует проход оптимизатора, который приводит все циклы к закрытой форме, для каждого определения, которое не соответствует закрытой форме он вставляет фи-функцию в выходе

цикла и заменяет использования переменной на ϕ -функцию. Во время структурных оптимизаций цикла оптимизатору останется лишь корректно добавлять аргументы соответствующим ϕ .

Для более общих копирующих трансформаций CFG переменные внутри копируемого участка все же выводятся из SSA-формы, а после трансформации SSA-свойство восстанавливается дополнительным проходом.

Алгоритмы построения SSA, рассмотренные выше, можно также адаптировать и для восстановления SSA-свойства.

1.2.5. Sea of nodes

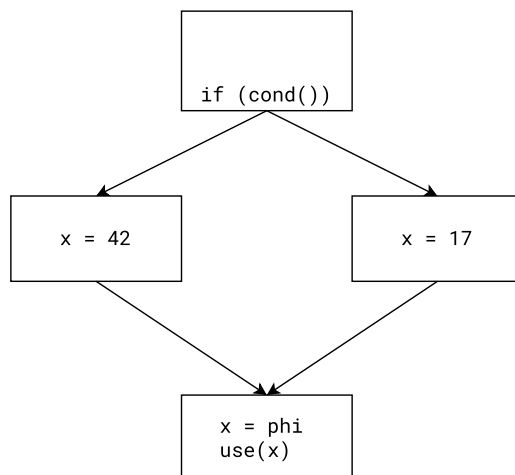
Sea of nodes (сокращенно SOF) - это развитие идей графа потока управления, но только в отличие от CFG в SOF не все инструкции имеют закрепленное положение в CFG, вместо этого они *плавают* в определенном интервале. Идея такого представления появилась из наблюдения что проходы оптимизатора, использующего классические CFG + SSA часто переупорядочивают некоторые инструкции, например выносят инвариантные вычисления из цикла. Вместо множества проходов, оптимизирующих положение инструкций внутри CFG, SOF не имеет четкого положения инструкций изначально и использует один алгоритм для определения положения всех плавающих инструкций.

Такое представление оказалось эффективным как по памяти, потребляемой оптимизатором, так и по времени компиляции. В настоящий момент SOF используется в серверном компиляторе Java, и компиляторе GraalVM.

1.3. Соответствие SSA и функциональным программированием.

Соответствие между SSA формой императивных программ и программ на функциональных языках было замечено в оригинальных статьях Эндрю Аппеля[14] и Ричарда Келси[15]. Действительно, требование единственного объявления переменной очень похоже на неизменяемость значений в функциональных языках программирования. А набор блоков можно рассматривать как набор функций, а фи-функции как параметры этих функций.

```
let entry = let bf_end = fun x -> use(x)
in let bf_true = bf_end(42)
in let bf_false = bf_end(17)
in if cond() then bf_true else bf_false
```



То есть любой CFG в SSA форме можно конвертировать в CPS или любое другое функциональное представление. Конверсия в другую сторону возможна не всегда из-за наличия в функциональных языках высшего порядка.

1.4. Thoring IR

Функциональные промежуточные представления изначально являются *текстовыми*, то есть промежуточное состояние программы - это тоже текст программы, выраженный в оптимизаторе в виде *абстрактного синтаксического дерева*. Например промежуточным представлением ghc является язык *core*. Компилятор использующий текстовое промежуточное представление изменяет напрямую АСД, что может быть неэффективно. Чтобы повысить эффективность компиляции функциональных программы были исследованы возможности графовых промежуточных представлений. В 2007-м компилятор sml/nj был переписан с использованием графового представления cps-термов.

В 2015-м году появился *Торин*, промежуточное представление совместившее идеи императивных и функциональных IR'ов. Торин базируется на стиле передаче продолжений термы которого представлены графом. В отличие от классических функциональных представлений функции в Торине не вложены явно друг в друга, вместо этого вложенность функций выражается при помощи def-use связей. Это удобно и эффективно поскольку Торин был создан для control-flow трансформаций кода в основном для элиминации функциональных замыканий.

Дизайн этого промежуточного представления позволяет использовать наиболее удобные концепции из обоих миров, функционального и императивного. Реализация Торина использует хорошо изученные структуры данных из императивных компиляторов, вроде дерева доминаторов и дерева вложенности циклов где это удобно, и также легко переходит к использо-

ванию ср.

2. ПОСТАНОВКА ЗАДАЧИ

Торин является удачным экспериментом по внедрению императивных элементов в чисто функциональное промежуточное представление. Целью этой работы является исследование возможностей обратного процесса - какую пользу может принести расширение Sea Of Nodes элементами CPS.

Одной из предполагаемых задач для исследования возможности внедрения стиля передачи продолжения в IR является быстрое восстановление SSA-свойства программы после применения движка копирующих трансформаций кода. Часть data-flow информации неявно сохраняется во время таких трансформаций для CPS, в отличие от SSA-формы. То есть можно перевести часть IR'а в CPS-форму, то есть избавиться от фи-функций, применить трансформацию, а затем локально перевести CPS обратно в SSA. Требуется исследовать эффективность такой конверсии относительно вывода части переменных из SSA-формы и последующего восстановления SSA-свойства программы.

Ещё одним из сценариев применения CPS является вынесение холодного кода в отдельные функции. При этом холодный код, который требуется вынести - это произвольный подграф в CFG, а не функция с одним входом и одним выходом.

На первый взгляд задачи могут выглядеть совершенной несвязанными, но у них есть нечто общее: в обеих задачах требуется выделить произвольный подграф в IR'е, произвести с ним действия, которые потенциально могут быть эффективнее выполнены в CPS и конвертировать этот граф обратно.

3. РАЗРАБОТАННЫЙ ПОДХОД

TODO

4. РЕЗУЛЬТАТЫ

TODO

ЗАКЛЮЧЕНИЕ

TODO

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Владимиров К. Оптимизирующие компиляторы. Структура и алгоритмы. Litres, 2024.
2. Rastello F., Tichadou F.B. SSA-based compiler design. Springer, 2022.
3. Click C., Paleczny M. A simple graph-based intermediate representation // ACM Sigplan Notices. ACM New York, NY, USA, 1995. Т. 30, № 3. С. 35–49.
4. Duboscq G. и др. Graal IR: An extensible declarative intermediate representation // Proceedings of the Asia-Pacific Programming Languages and Compilers Workshop. 2013. С. 1–9.
5. Jones S.L.P., Santos A.M. A transformation-based optimiser for Haskell // Science of computer programming. Elsevier, 1998. Т. 32, № 1-3. С. 3–47.
6. Appel A., Jim T. Continuation-passing style, closure-passing style // 16th Ann. ACM Symp. on Principles of Programming Languages. 1989.
7. Adams N. и др. Orbit: An optimizing compiler for Scheme // ACM SIGPLAN Notices. ACM New York, NY, USA, 1986. Т. 21, № 7. С. 219–233.
8. LLVM Project. LLVM Language Reference Manual. 2025.
9. Gosling J. и др. The Java Language Specification, Java SE 8 Edition. Addison-Wesley, 2015.
10. Evans B. Understanding Java method invocation with invokedynamic // Java Magazine. 2021.

11. Cytron R. и др. Efficiently computing static single assignment form and the control dependence graph // ACM Transactions on Programming Languages and Systems (TOPLAS). ACM New York, NY, USA, 1991. Т. 13, № 4. С. 451–490.
12. Click C.N. Combining Analyses, Combining Optimizations: phdthesis. Houston, Texas: Rice University, 1995.
13. Braun M. и др. Simple and efficient construction of static single assignment form // International Conference on Compiler Construction. Springer, 2013. С. 102–122.
14. Appel A.W. SSA is functional programming // Acm Sigplan Notices. ACM New York, NY, USA, 1998. Т. 33, № 4. С. 17–20.
15. Kelsey R.A. A correspondence between continuation passing style and static single assignment form // ACM SIGPLAN Notices. ACM New York, NY, USA, 1995. Т. 30, № 3. С. 13–22.

ПРИЛОЖЕНИЕ

П.1. Первая глава приложения

TODO